

Apache Beam & SparkML

Alka Bhushan

Apache Beam <https://beam.apache.org/>

- Features

- Unified programming model for batch and stream processing
- Can be executed locally for development, testing, and debugging using small datasets
- Can execute the same pipeline on multiple processing engines (runners)
 - Apache Flink, Apache Spark, Google Cloud DataFlow, Samza
- Supports multiple programming languages
 - Python, Java, Go, YAML, SQL

- Usefulness

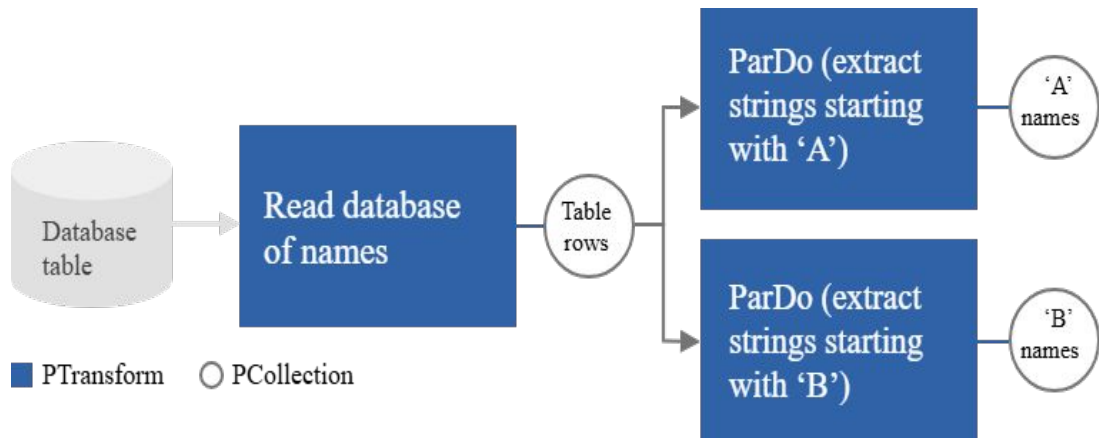
- **Portability across processing engines** without significant changes to business logic.
- **Ability to run on different cloud and on-premises environments** through supported runners.
- **Advanced event-time processing capabilities**, including watermarks, windows, and triggers.
- **Single API for both batch and streaming pipelines**, independent of the execution engine.
- **Suitable for building reusable and maintainable data processing platforms**, rather than tightly coupling code to a specific engine.
- **Reduces vendor and engine lock-in** by separating pipeline logic from the execution environment.

Beam: Basics

- Pipeline
- PCollection
- PTransform
- Aggregation
- User-defined Function (UDF)
- Schema
- SDK
- Runner
- Window
- Watermark
- Trigger
- State and Timers
- Splittable DoFn

Pipeline

- **Pipeline as a Directed Acyclic Graph (DAG):** Represents the complete data processing workflow, including reading data, applying transformations (**PTransforms**), and writing outputs. Data flows between transformations as **PCollections**.
- **Unified Processing for Batch and Streaming:** Beam pipelines can process both **bounded (batch)** and **unbounded (streaming)** data using the same programming model, enabling portable execution across multiple runners.



PCollection (Core Data Structure in Beam)

- **Distributed Dataset/Stream:** A PCollection is an unordered, potentially distributed collection of data elements processed in parallel by Beam transforms.
- **Supports Batch & Streaming:** Can be **bounded** (fixed-size dataset for batch processing) or **unbounded** (continuously growing stream for real-time processing).
- **Time-Aware Processing:** Every element has a **timestamp**, belongs to a **window**, and is tracked using **watermarks** to enable event-time processing and handle late-arriving data.
- **Portable Data Representation:** Each PCollection uses a **Coder** (serialization format) and a **Windowing Strategy** that governs grouping, triggering, and aggregation behavior.

PTransform (Data Processing Operation)

- **Represents Processing Logic:** A PTransform defines a processing step in a pipeline, taking one or more **PCollections** as input and producing one or more **PCollections** as output.
- **Parallel Execution:** User-defined logic is applied to elements of a PCollection, and Beam executes this logic in parallel across multiple workers for scalable processing.
- **Built-in and Custom Transforms:** Beam provides core transforms such as **ParDo**, **GroupByKey**, **Combine**, and **Count**, while also allowing developers to create reusable custom composite transforms.
- **Types of Transforms:** Includes **Source** (Read/Create), **Processing** (ParDo, Combine, GroupByKey), **Output** (Write), and **User-defined Composite** transforms.

Basic Program

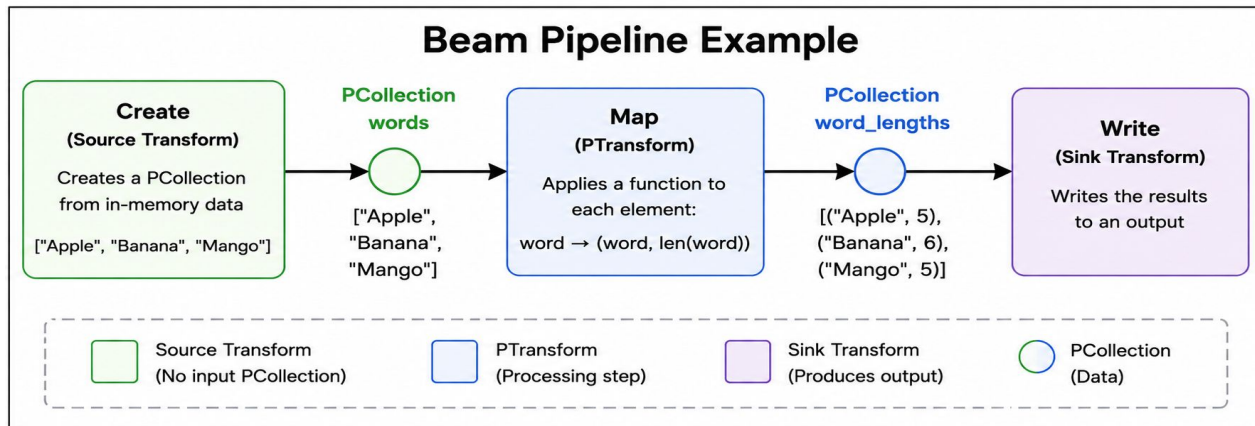
```
import apache_beam as beam
```

```
with beam.Pipeline() as p:
```

```
    words = p | beam.Create(["Apple", "Banana", "Mango"])
```

```
    word_lengths = words | beam.Map(lambda word: (word, len(word)))
```

```
    word_lengths | beam.Map(print)
```



Aggregation

- **Aggregation combines multiple elements into a single result** by grouping data with the same **key and window** and applying operations such as **Count, Sum, Max, or Average**.
- **Key aggregation transforms:**
 - **GroupByKey** – groups all values for the same key. Output is no smaller than input.
 - **CombinePerKey** – groups and applies a combine function (e.g., sum, count) to produce a smaller result set.
- **For streaming data**, aggregation relies on **Windowing, Watermarks, and Triggers** to determine when results are complete and should be emitted.
- **Parallel Scalability:** Aggregations scale efficiently across millions of keys/windows; when keys are few, they can be split into sub-keys and later recombined to increase parallelism without changing the result.

UDFs and Schemas

User-Defined Functions (UDFs)

- Custom user code that defines how Beam processes data.
- Examples: **DoFn** (processing), **CombineFn** (aggregation), **WindowFn** (windowing).
- Executed in parallel across distributed workers.

Schemas

- Define the structure of data in a PCollection using **named fields and data types**.
- Enable easier querying, transformations, and **Beam SQL** support.

Advanced Concepts

- **Windowing**
 - Divides unbounded data streams into finite chunks for processing.
 - Supports Fixed, Sliding, Session, and Global windows.
- **Watermarks**
 - Estimate how complete the incoming event-time data is.
- **Triggers**
 - Control when window results are emitted.
 - Support early, on-time, and late result generation.
- **State**
 - Maintains per-key information across events
- **Timers**
 - Schedule event-time or processing-time actions.
- **Splittable DoFn (SDF)**
 - Enables dynamic splitting and parallel execution of large workloads.

Links

- Documentation: <https://beam.apache.org/documentation/>
- <http://beam.apache.org/get-started/wordcount-example/>

Beam Demo

Spark ML <https://spark.apache.org/>

Spark ML provides tools such as:

- ML Algorithms: common learning algorithms such as classification, regression, clustering, and collaborative filtering
- Featurization: feature extraction, transformation, dimensionality reduction, and selection
- Pipelines: tools for constructing, evaluating, and tuning ML Pipelines
- Persistence: saving and load algorithms, models, and Pipelines
- Utilities: linear algebra, statistics, data handling, etc.

Basic Statistics : **correlation** (Pearson's and Spearman's correlation), **hypothesis testing** (currently supports `ChiSquareTest`), **summarizer** (available metrics are the column-wise max, min, mean, sum, variance, std, and number of nonzeros, as well as the total count).

Data Sources: Parquet, CSV, JSON and JDBC , others: image and libsvm

<https://spark.apache.org/docs/latest/ml-statistics.html> , <https://spark.apache.org/docs/latest/ml-datasource.html>

ML Pipelines : Main Concepts

ML pipelines provide high level APIs built on top of DataFrames to create and tune machine learning pipelines.

- DataFrame : uses dataframe from Spark SQL as an ML dataset
 - Can have different columns such as text, feature vectors, true labels, and predictions
- Transformer : an algorithm which can transform one DataFrame into another DataFrame
 - An ML Model
- Estimator : an algorithm which can be fit on a DataFrame to produce a Transformer
 - A learning algorithm
- Pipeline : chains multiple transformers and estimators to build a ML workflow
- Parameter : transformers and estimators share a common API for specifying parameters

<https://spark.apache.org/docs/latest/ml-pipeline.html>

Pipeline Components and Properties

Components

- Transformers
 - Takes input data and produces transformed output using either feature transformation methods or trained models.
 - Converts dataframe into another dataframe by appending one or more columns
 - E.g. text -> feature vectors, trained model -> predicted labels
- Estimators
 - Learns from input data and produces a trained model

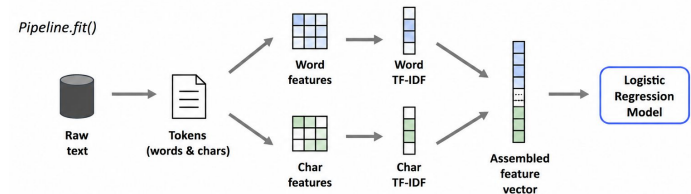
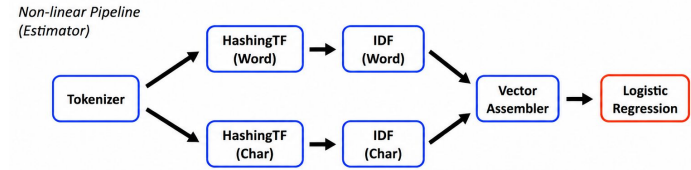
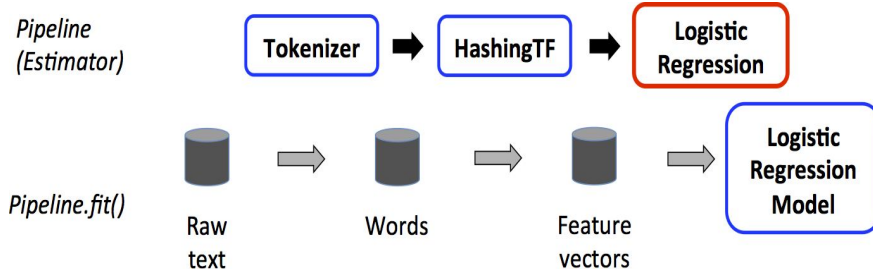
Properties

- `Transformer.transform()` and `estimator.fit()` are stateless
- Each instance of Transformer or estimator has a unique id

Pipeline : A Sequence of Pipeline Stages

Properties

- DAG pipelines
 - Linear pipelines : each stage uses data produced by the previous stage
 - Non-Linear pipelines : each stage uses data produced by one or more previous stages



Pipeline : A Sequence of Pipeline Stages

Properties

- Runtime checking
 - Since Pipelines can operate on DataFrames with varied types, they use schema based run time checking instead of compile-time type checking.
- Unique Pipeline Stages
 - A Pipeline cannot use the same stage object twice because each stage must have a unique ID. Different objects of the same type are allowed.

Parameters

- MLlib uses a uniform parameter API where parameters can be set directly on an Estimator/Transformer instance or passed via a ParamMap to `fit()/transform()`.
 - E.g. `lr.setMaxIter(10)` sets a parameter for one instance, while `ParamMap(lr1.maxIter→10, lr2.maxIter→20)` allows different values for different instances.

ML Persistence : Saving and Loading Pipelines

- Spark ML supports saving and loading models and Pipelines across Scala, Java, and Python using ML persistence.
- A model saved in Spark 3.x can typically be loaded in later minor/patch releases with compatible behavior.

Model Selection and Tuning

- Tuning can be done for individual element in the pipeline, or for entire pipelines
 - Single Estimator or entire Pipeline
- Cross Validator
 - Similar to GridSearchCV
 - Split data into k-folds
 - For each **parameter combination**:
 - Train on **k-1 folds**
 - Validate on remaining fold
 - Compute **average metric**
 - Choose **best parameters**
 - Retrain on **full dataset**
 - Parallel hyperparameter evaluation: `cv.setParallelism(4)`
 - Concurrent evaluations each logically use the full dataset partitions, but they share the same cluster resources during execution.

Additional Spark ML Topics (Guide References)

Extracting, Transforming & Selecting Features

<https://spark.apache.org/docs/latest/ml-features.html>

Classification & Regression

<https://spark.apache.org/docs/latest/ml-classification-regression.html>

Clustering

<https://spark.apache.org/docs/latest/ml-clustering.html>

Collaborative Filtering

<https://spark.apache.org/docs/latest/ml-collaborative-filtering.html>

Frequent Pattern Mining

<https://spark.apache.org/docs/latest/ml-frequent-pattern-mining.html>

Advanced Topics

<https://spark.apache.org/docs/latest/ml-advanced.html>

Code Demo